

Desert Storm: 1991

Technical Document

Pixelforge Studios

Vladislav Zolotukhin

Anthony Korepanov

Yevhen Chernytskyi

Lisa Kaur

Unity 2022.3.62f1 | Version 1.0

Table of Contents

1. Project Overview.....	3
2. System Architecture.....	4
2.1 Unity Editor Version.....	4
2.2 Scene Structure.....	4
2.3 Room Prefab Architecture	5
2.4 Procedural Generation System	6
2.5 Asset Organization	6
3. Development Environment Setup	7
3.1 Prerequisites	7
3.2 Optional Developer Tools (Windows Only)	7
3.3 Cloning the Repository	7
3.4 Opening the Project in Unity.....	7
3.5 Required Unity Packages.....	8
3.6 Layer Configuration	8
3.7 Running the Game in the Editor	8
4. Git Workflow.....	9

4.1 Keeping Your Local Repository Up to Date.....	9
4.2 Creating a Feature Branch	9
4.3 Committing Work.....	9
4.4 Pushing a Branch to Remote	9
4.5 Opening a Pull Request	9
4.6 Review and Merge	10
5. Key Technical Decisions.....	11
5.1 ScriptableObject-Driven Architecture	11
5.2 Universal Render Pipeline (URP).....	11
5.3 Cinemachine	12
5.4 A* Pathfinding for Enemy Navigation	12
6. Known Issues and Deviations from Original Scope	13
6.1 Removed Functional Requirements	13
7. Running and Building the Game	14
7.1 Running the Pre-Built Executable (End Users)	14
7.2 Building the Game from Source (Developers).....	14
7.3 Build Targets	14

1. Project Overview

Desert Storm: 1991 is a 2D top-down military roguelike developed in Unity, targeting Windows, Android, and iOS platforms as a free-to-play release. The game blends fast-paced tactical shooter gameplay with roguelike progression mechanics, class-based character selection, procedural level generation, and strategic resource management.

The project addresses a recognized gap in the 2D indie game market: while roguelike dungeon crawlers (e.g. The Binding of Isaac, Enter the Gungeon) and military shooters are individually popular among the 18-35 demographic, few titles combine both. Desert Storm: 1991 targets this underserved niche by applying the replayability and strategic depth of roguelikes to a historical military setting.

Core gameplay features in scope for v1.0 include:

- Character class selection and appearance customization
- Multi-weapon combat with distinct ammunition types and behaviors
- Diverse enemy types with different combat roles and navigation behaviors
- Procedurally generated levels with randomized objectives and loot
- An in-game store, HUD, minimap, and full dungeon overview map
- Save and load state persistence across sessions
- Configurable audio and visual settings

Multiplayer, a level editor, and advanced adaptive enemy AI are explicitly out of scope for this version.

2. System Architecture

2.1 Unity Editor Version

Desert Storm: 1991 was developed using Unity 2022.3.62f1 (LTS) with the Universal Render Pipeline (URP). The project targets Windows, Android, and iOS platforms and uses a 2D top-down camera perspective rendered via Cinemachine.

2.2 Scene Structure

The project is organized into two primary scenes and several subordinate menu scenes.

MainMenuScene serves as the application entry point. Its hierarchy contains a Main Camera, the main menu UI canvas, an EventSystem, the game logo GameObjects, a screen cursor with an AspectRatioFitter, a background with a panning slider script for a dynamic animated background, and a MusicManager. From here, the player navigates to one of several child menu scenes:

- LogoScene — Displays the studio logo via a Video Player and Camera on launch.
- CharacterSelectorScene — Allows the player to choose a character class before starting a new run. Contains a CharacterSelector with a SelectorBackground, SpriteMask, CharacterSelectorAnchor, and CharacterTextCanvas.
- LoadGameScene — Presents up to three save slots, each displaying character class thumbnail, level reached, and save date, with Load and Delete controls. Driven by a SaveManager.
- HighScoreScene — Displays a scrollable high scores table with rank, name, level, and score columns via a HighScoreManager and a scrollable DisplayHighScoresUI.
- InstructionsScene — A static informational screen displaying game controls and HUD element icons (ammo, minimap, chest, weapons, etc.).
- OptionsScene — Contains music and sound volume sliders with increment/decrement controls and a full key bindings reference panel with directional icons for each action.

MainGameScene contains the core gameplay systems. Its hierarchy includes:

- **Camera System** — Camera System

A Main Camera paired with a Cinemachine Virtual Camera (CM vcam MainCamera) and a CinemachineTargetGroup that follows the player. A CursorTarget GameObject allows the camera to pan toward the mouse cursor, widening the player's field of view dynamically.

- **Rendering** — Rendering

A PostProcessing volume and a Global Light 2D for URP lighting control.

- **GameManager** — GameManager

Central coordinator for game state.

- **HighScoreManager** — HighScoreManager

Tracks and persists score data across sessions.

- **DungeonBuilder** — DungeonBuilder

Drives the procedural level generation system (detailed in Section 2.4).

- **Minimap** — Minimap

Contains its own dedicated camera, lighting, and minimap player representation. Renders exclusively from Tilemap6_Minimap layer tiles. Supports both the HUD-embedded minimap and the full-screen dungeon overview map toggled via the Tab key (DungeonOverviewMap).

- **UI** — UI

A compound UI hierarchy containing the screen cursor, HUD, pause menu, save slots overlay, in-game options menu, transition canvas, and fade screen.

- **PoolManager** — PoolManager

Manages object pooling for performance-critical entities such as projectiles and enemies.

- **Service Objects** — SaveManager, SoundEffectManager, EnemySpawner, MusicManager

Scene-level service objects handling persistence, audio, and enemy lifecycle.

2.3 Room Prefab Architecture

Each room in the game is represented by two paired assets: a Prefab and a ScriptableObject.

The prefab defines the room's visual and physical structure through a layered tilemap hierarchy under a parent Grid GameObject:

Layer	Tilemap	Purpose
1	Tilemap1_Ground	Floor tiles
2	Tilemap2_Decoration1	Sprite-plane decorations (same depth as player/enemies)
3	Tilemap3_Decoration2	Additional sprite-plane decorations
4	Tilemap4_Front	Foreground tiles rendered in front of the player to simulate depth (e.g. rooftops)
5	Tilemap5_Collision	Invisible collision and A* pathfinding obstacle layer
6	Tilemap6_Minimap	Invisible in-game; exclusively used to render room shape on the minimap

A BoxCollider2D on the prefab root defines the room's occupied bounds. Rooms may optionally include a ChestSpawner component, which spawns a loot chest upon room clearance with a configurable probability and item pool (health, ammo, or weapons).

The accompanying ScriptableObject defines:

- **Exit configurations** — Each room supports up to four exits (N/S/E/W). The script programmatically covers unused exits at runtime by copying an adjacent wall tile section (defined by a source tile coordinate plus width/height parameters) across all six tilemap layers, allowing a single prefab to appear in multiple layout configurations.

- Combat parameters — Enemy spawn coordinates, enemy pool composition, spawn rate, simultaneous enemy cap, and randomized count ranges (e.g. 10-12 enemies drawn from a defined pool).
- Room bounds — Upper and lower boundary coordinates used by the dungeon builder.
- Door positions — Doors are placed at exit centers, are player-interactable, and lock automatically during active combat to prevent the player from fleeing a room mid-encounter.

2.4 Procedural Generation System

The dungeon builder generates each level using a two-layer randomization system. A dungeon level ScriptableObject defines a pool of node graphs, each representing a possible level layout. At runtime, one node graph is selected and can be randomly rotated for additional variation.

Each node in the graph represents either a room or a corridor. The dungeon builder resolves each node by selecting randomly from a pool of eligible room prefabs assigned to that node type (small, medium, large, item room, or boss room). Corridors are oriented as either North-South or East-West, determined automatically based on the relative positions of the rooms they connect. This system ensures that no two playthroughs produce the same level layout while maintaining structural coherence between rooms.

2.5 Asset Organization

The project asset hierarchy is organized as follows:

- Enemy/ — ScriptableObjects per enemy type (Commander, Elite_HMG, Elite_RPG, Medic_PPSH, Sniper_Dragunov, Standard_AK, Standard_Pistol), each containing EnemyDetails and MovementDetails assets.
- Player/ — Player prefab and related assets.
- Sounds/ — Audio clips and SFX assets.
- Weapons/ — Weapon assets organized by faction (Enemy, Player), each containing AmmoDetails, AmmoHitEffect, SoundEffect, WeaponDetails, and WeaponShootEffect ScriptableObjects.
- Rendering/ — URP rendering assets and post-processing profiles.
- Scenes/MainGameScene/ — Global Volume Profile and Post Processing Profile.
- Scenes/MainMenu/ — All subordinate menu scene files (CharacterSelectorScene, HighScoreScene, InstructionsScene, LoadGameScene, LogoScene, OptionsScene).
- ScriptableObjectAssets/Bosses/ — Boss-specific ScriptableObjects.
- ScriptableObjectAssets/Dungeon/DungeonRoomNodeGraph/ — Node graph assets organized by level (Level1-Level4), each containing RoomNodeGraph and VZ_NodeGraph variants.
- Minimap/, UI/, Prefabs/ — Supporting assets for HUD, menus, and reusable GameObjects.

3. Development Environment Setup

This section outlines the steps required to configure a local development environment and open the Desert Storm: 1991 project in the Unity Editor.

3.1 Prerequisites

The following tools must be installed before opening the project:

- Unity Hub — The Unity project and editor version manager. Download from <https://unity.com/download>.
- Unity Editor 2022.3 LTS — Must be installed through Unity Hub. The project was developed and tested on version 2022.3.62f1 specifically; other 2022.3.x patch versions should be compatible but are not guaranteed.
- Visual Studio 2022 — The recommended IDE for C# script editing. During installation via the Visual Studio Installer, ensure the Game development with Unity workload is selected.
- Git — Required to clone the repository and follow the project's branch-based workflow.

3.2 Optional Developer Tools (Windows Only)

The following Visual Studio extensions are not required but are recommended for an improved development experience on Windows:

- CodeMaid Spade — Code cleanup and formatting.
- Semantic Colorizer — Enhanced syntax highlighting for improved code readability.

3.3 Cloning the Repository

With Git installed, clone the repository to your local machine:

```
git clone https://github.com/<org>/Desert-Storm-1991.git
```

Navigate into the project directory after cloning:

```
cd Desert-Storm-1991
```

3.4 Opening the Project in Unity

1. Launch Unity Hub.
2. Click Open and navigate to the cloned Desert-Storm-1991 directory.
3. Unity Hub will detect the required editor version (2022.3 LTS). If it is not yet installed, Unity Hub will prompt you to install it — accept and allow the installation to complete before proceeding.
4. Once the correct editor version is confirmed, open the project. The initial import may take several minutes as Unity processes and compiles all assets and packages.

5. If prompted about safe mode or compilation errors on first open, dismiss and allow Unity to fully load before taking action.

3.5 Required Unity Packages

The following packages are used by the project and are defined in the project's Packages/manifest.json. They should resolve automatically when the project is first opened in Unity; no manual installation should be required:

- 2D Sprite
- 2D Tilemap Editor
- 2D Tilemap Extras
- Cinemachine
- TextMeshPro
- Unity UI
- Universal Render Pipeline (URP)
- Visual Scripting
- Visual Studio Editor

If any packages fail to resolve, open the Package Manager via Window > Package Manager and install any missing packages manually from the Unity Registry.

3.6 Layer Configuration

The project uses custom physics and rendering layers that must be present for the game to function correctly. To verify or restore layer configuration, navigate to Edit > Project Settings > Tags and Layers in the Unity Editor and confirm the expected layers are present. These are version-controlled as part of the project's ProjectSettings/TagManager.asset file and should be restored automatically upon cloning.

3.7 Running the Game in the Editor

1. In the Project panel, navigate to Assets/Scenes and open MainGameScene or MainMenuScene.
2. Press the Play button in the Unity toolbar to enter Play Mode.
3. To test from the main menu entry point, open MainMenuScene first, as it manages scene navigation to all other scenes.

4. Git Workflow

All development on Desert Storm: 1991 follows a branch-based Git workflow centered on a protected main branch. Direct commits to main are not permitted; all changes are introduced through feature branches and reviewed via pull requests before merging.

4.1 Keeping Your Local Repository Up to Date

Before beginning any new work, ensure your local main branch reflects the latest state of the remote repository:

```
git checkout main
git pull origin main
```

This prevents merge conflicts and ensures new branches are created from the most current codebase.

4.2 Creating a Feature Branch

Create a new branch for each discrete feature or bug fix. Branch names should follow the feature/ or fix/ prefix convention and be descriptive of the work being done:

```
git checkout -b feature/new-enemy-ai
```

4.3 Committing Work

Stage and commit changes incrementally with clear, descriptive commit messages that summarize what was changed and why:

```
git add .
git commit -m "Implement basic enemy AI movement"
```

Avoid large monolithic commits — prefer smaller commits scoped to a single logical change where possible.

4.4 Pushing a Branch to Remote

Once local work is ready for review, push the branch to the remote repository on GitHub:

```
git push origin feature/new-enemy-ai
```

4.5 Opening a Pull Request

1. Navigate to the repository on GitHub.

2. GitHub will detect the recently pushed branch and display a Compare & pull request prompt — click it.
3. Ensure main is selected as the target (base) branch.
4. Write a short but informative description of the changes introduced by the branch.
5. Submit the pull request for team review.

4.6 Review and Merge

At least one other team member should review the pull request before it is merged. Once approved, merge into main using GitHub's Merge Pull Request button. Do not merge your own pull requests without peer review except in exceptional circumstances.

After merging, delete the remote branch to keep the repository clean:

```
git branch -d feature/new-enemy-ai  
git push origin --delete feature/new-enemy-ai
```

Branches may be retained if they represent a significant feature and serve ongoing documentation or reference value.

5. Key Technical Decisions

This section documents the rationale behind the major architectural and tooling decisions made during development. Given a four-month development window with a team new to Unity game development, decisions were evaluated primarily on feasibility, development speed, and alignment with the game's core design pillars of modularity, replayability, and cross-platform support.

5.1 ScriptableObject-Driven Architecture

The most significant architectural decision in the project is the use of Unity ScriptableObjects as the primary data container for rooms, enemies, weapons, and dungeon levels rather than embedding configuration directly in MonoBehaviour scripts or scene hierarchies.

This decision was driven by several factors specific to Desert Storm: 1991's design. The game is built around modularity and procedural variation, meaning the same room prefab may appear multiple times within a single level, in different positions, with different exit configurations, and surrounded by different adjacent rooms. Rather than duplicating prefabs to account for these variations, a single prefab is paired with a single shared ScriptableObject asset that defines its invariant properties: enemy spawn coordinates and pool, chest spawner parameters, combat configuration, room bounds, and door positions. Exit geometry is resolved at runtime by the dungeon builder, meaning the prefab itself never needs to be duplicated to represent a different configuration.

This pattern also established a clean and consistent workflow for the team: a designer first authors the room prefab in the editor, then configures its accompanying ScriptableObject to define its runtime behavior. This separation of visual construction from behavioral configuration reduced the likelihood of errors and made it straightforward for multiple team members to work on different rooms simultaneously without conflict.

ScriptableObjects also provide globally accessible shared state across scenes without requiring objects to persist between scene loads via DontDestroyOnLoad. This was particularly valuable for the save state system, which needs reliable access to a consistent list of spawned rooms, their cleared states, enemy populations, and loot — all of which must be reproduced exactly when a player loads a saved game. Since ScriptableObject assets are persistent references rather than scene-bound instances, the save manager can reliably reference and serialize this data regardless of which scene is currently loaded.

Finally, this approach proved more memory efficient than alternatives: a single ScriptableObject asset is loaded once and referenced by any number of prefab instances, rather than each instance maintaining its own copy of the same configuration data.

5.2 Universal Render Pipeline (URP)

The project uses Unity's Universal Render Pipeline rather than the default Built-in Render Pipeline. For a 2D top-down game, URP offers a more optimized and lightweight rendering path that avoids the overhead of features irrelevant to this project's visual style. This is particularly

significant given the intended deployment targets of Android and iOS, where rendering performance and battery efficiency are constrained compared to desktop platforms. URP's mobile-friendly design made it the appropriate choice to ensure the game maintains stable performance across all target platforms without requiring platform-specific rendering configurations.

5.3 Cinemachine

Rather than implementing custom camera tracking and interpolation logic from scratch, the project uses Unity's Cinemachine package to handle all camera behavior. Cinemachine provides the player-following virtual camera, smooth interpolation, the CinemachineTargetGroup configuration that offsets the camera toward the mouse cursor to extend the player's visible field of view, and the boundary constraints that prevent the camera from panning outside room bounds. Implementing equivalent behavior manually would have represented a significant time investment with no design upside, making Cinemachine the practical choice given the project's four-month timeline.

5.4 A* Pathfinding for Enemy Navigation

Enemy navigation uses an A* (A-star) pathfinding algorithm operating on the collision tilemap layer (Tilemap5_Collision). A* was selected because it provides reliable and efficient grid-based pathfinding sufficient for the game's combat design without requiring a more complex navigation solution. Desert Storm: 1991's rooms are intentionally detailed with tactical cover objects, meaning enemies must navigate around obstacles to reach the player rather than moving in a straight line. A* handles this naturally. More sophisticated navigation or adaptive AI behavior was explicitly descope'd for v1.0 as it fell outside the complexity feasible within the development timeline, and A* alone produces engaging enough enemy behavior to support the intended tactical gameplay.

6. Known Issues and Deviations from Original Scope

Due to the limitations of the available resources, time constraints, and loss of a team member, there was a reduction in the original scope of the project which led to several functional requirements being removed or adjusted to be reduced in scope.

6.1 Removed Functional Requirements

1. FR2 – In-game vendor system

The system allowing players to purchase weapons and healing items using in-game currency was not implemented. As a result, no in-game economy or vendor system exists. The in-game store was instead replaced with an item room allowing the player to obtain weapons or resources from a pool of randomized items that varies on a per-level basis. Unlike other chest item spawners, there is a guaranteed chance of the player obtaining a chest in this room and it spawns immediately upon entry. The intended functionality is to provide a lower bound to the progression system of the game, enabling the player to obtain a weapon drop (with a low probability of obtaining a more powerful weapon that would otherwise be unavailable until later in the game progression) and ensure that a minimum baseline of progression is maintained if the player is not able to upgrade their arsenal otherwise as the item room is guaranteed to appear once in every single node graph of a level.

2. FR3 – Difficulty selection system

The option for players to change difficulty through the settings menu was not included. Difficulty is instead managed through fixed balancing and progression design.

3. FR16 – Environmental interaction

Mechanics Interactive environmental elements such as traps, switches, and mechanisms affecting enemy movement or unlocking areas were not implemented. However, rudimentary environment interaction exists in the form of chests which can be interacted with and opened to obtain items or pushed around, and doors which can be interacted with to open and allow the player to progress to new areas.

4. FR17 – Mission objective tracking system

A system for displaying and tracking mission objectives via the HUD or pause menu was not included. The game instead uses a progression-based structure without explicit objective tracking. The mission objective of every level is currently to clear the level of all enemies and to defeat the boss.

5. FR20 – Character appearance customization

The ability for players to customize character appearance or uniforms through a settings menu was not implemented. This is also a consequence of removing the microtransaction-based cosmetic system economy. Currently, character appearances vary only by class selection.

6. FR21 – Microtransaction-based cosmetic system

The proposed system for purchasing character skins using microtransactions was not included in the final implementation.

7. Running and Building the Game

7.1 Running the Pre-Built Executable (End Users)

A pre-built Windows executable is provided in the project's releases. To run the game, navigate to the releases page [here](#) to obtain the latest build, download and extract the contents, then double-click DesertStorm1991.exe. No additional installation, Unity Editor, or development tooling is required to run the pre-built executable.

7.2 Building the Game from Source (Developers)

Developers who wish to generate a new build from source must first complete all environment setup steps described in Section 3. Once the project is open in the Unity Editor, a new build can be produced as follows:

1. Navigate to File > Build Settings in the Unity Editor toolbar.
2. Confirm the target platform is set to Windows, Mac, Linux. To switch platforms, select the desired target and click Switch Platform — note that a platform switch may trigger a reimport of project assets and take several minutes.
3. Ensure all required scenes are present and correctly ordered in the Scenes In Build list. The scene order should be: LogoScene, MainMenuScene, followed by the subordinate menu scenes, and MainGameScene last.
4. Click Build to select an output directory and generate the executable without launching it, or click Build and Run to generate the build and immediately launch the game upon completion.
5. The output directory will contain DesertStorm1991.exe along with the required _Data/ folder and supporting runtime libraries. All of these files must be kept together — the executable cannot be run in isolation without its accompanying output files.

7.3 Build Targets

The project is currently configured and tested for Windows. Android and iOS are intended deployment targets for future releases. To build for Android or iOS, the corresponding build support modules must be installed via Unity Hub (Installs > Add Modules) and the platform switched in Build Settings accordingly. Additional platform-specific configuration, such as Android SDK setup or Xcode requirements for iOS, will be required and is outside the scope of this document for v1.0.